

Lecture 4: Assembly

Professor G. Sandoval

04



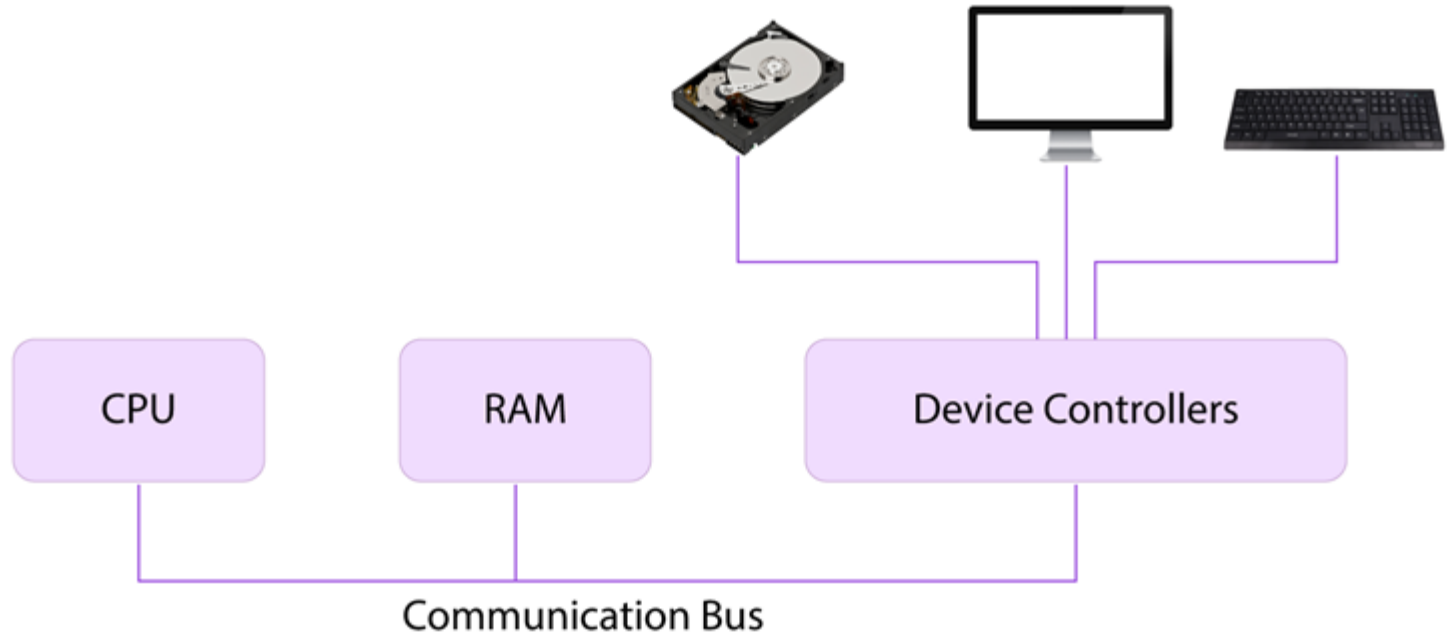
Some slides adapted by G. Sandoval for CS3224, from slide by Brendan Dolan-Gavitt and Bryant and O'Hallaron, Computer Systems: A programmer's Perspective, Third Edition

The Original IBM PC : 1984



PC Architecture

(Vastly simplified)



PC Architecture

(slightly less simplified)

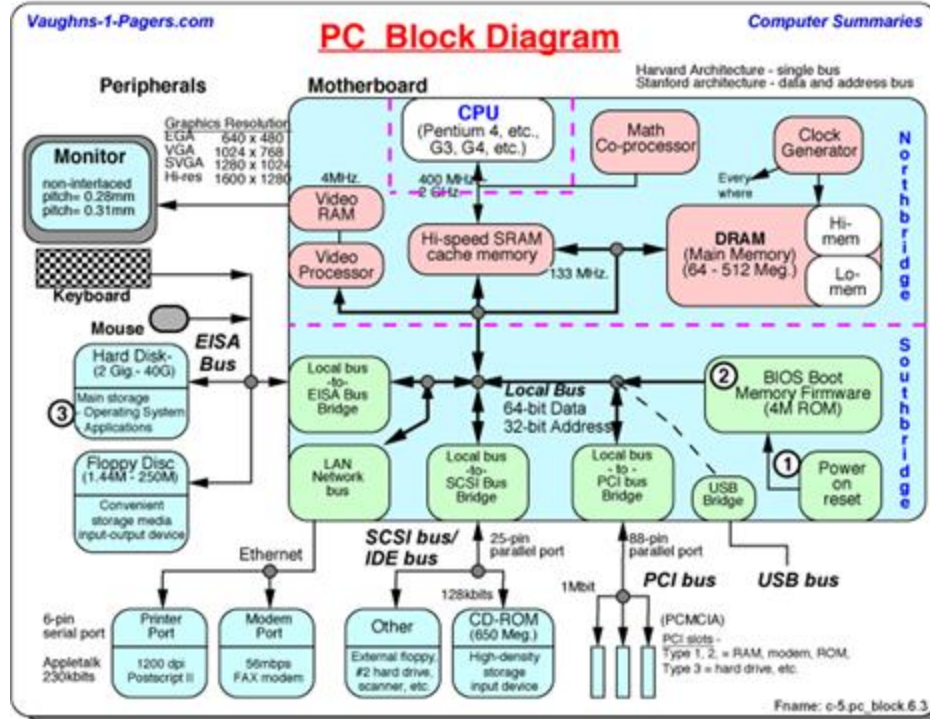
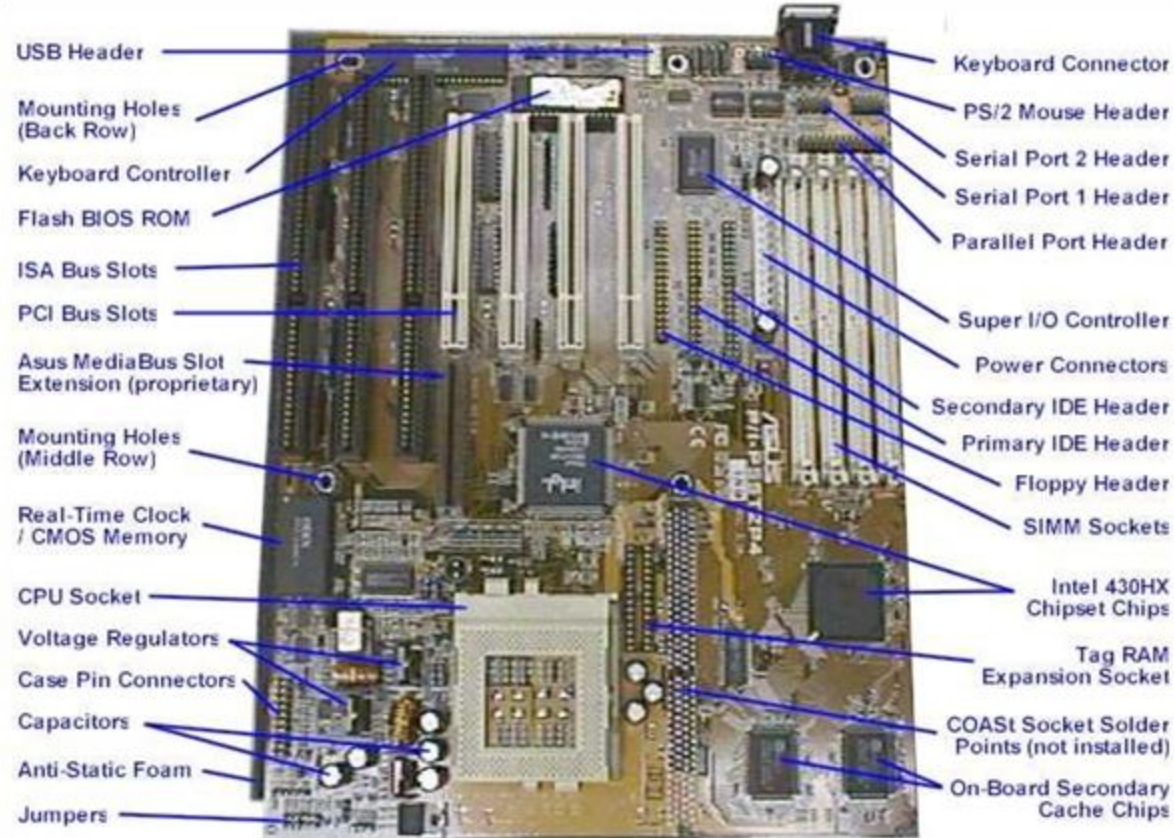


Diagram by Vaughn Aubuchon

PC Architecture



2015 State of the Art

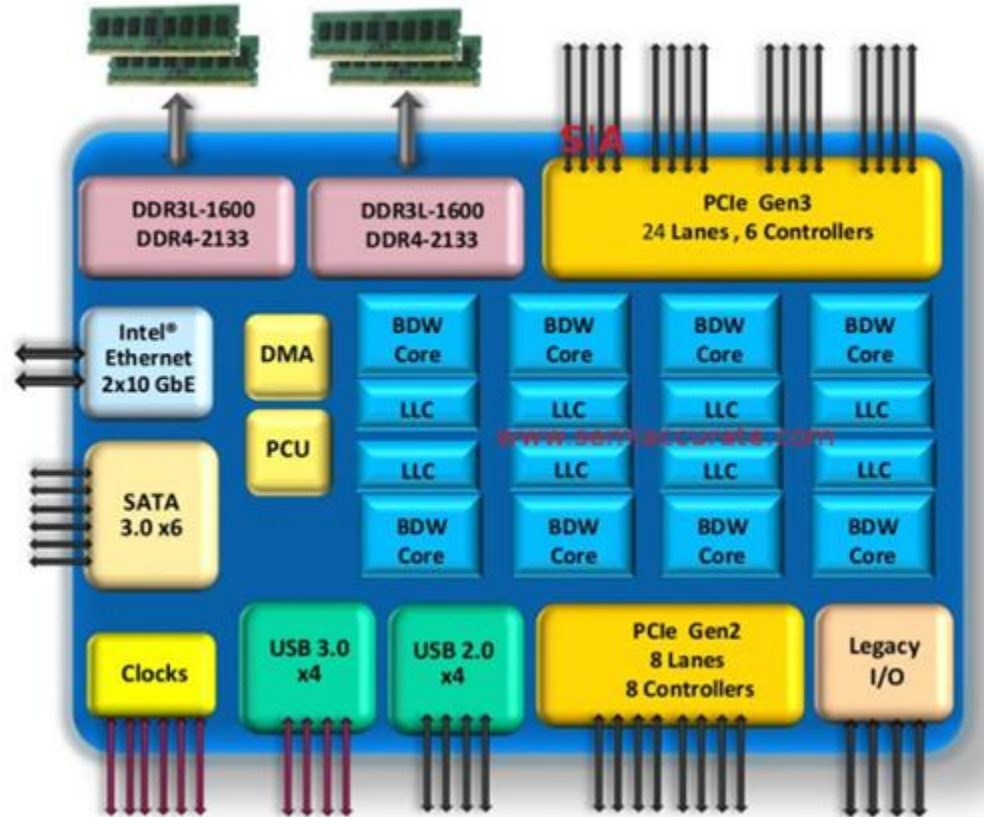
- Core i7 Broadwell 2015

Desktop Model

- 4 cores
- Integrated graphics
- 3.3-3.8 GHz
- 65W

Server Model

- 8 cores
- Integrated I/O
- 2-2.6 GHz
- 45W



IBM PC (1984) -> Core i7(2015) -> 2025

- **Baseline (1984 IBM PC / PC-AT)**

- CPU: Intel 8088 @ **4.77 MHz** (5150) or 80286 @ **6–8 MHz** (PC/AT).
- RAM: **256–640 KB**. Storage: 20 MB HDD standard on AT.
- 8088 transistor count: **~29,000**.

- **2015 reference (Broadwell i7-5775C)**

- Cores: **4/8, 2-2.6 GHz** turbo. ~1.9 billion transistors

- **2025 reference (mainstream desktop)**

- Example: Intel **Core i9-14900K** — **24 cores (8P+16E)**, turbo **up to 6.0 GHz**.
- Modern NVMe SSDs: **~7 GB/s** sequential reads (PCIe 4.0; PCIe 5.0 can exceed this).
- Modern SoC transistor counts (for a feel of integration): Apple **M4 ~28 B** transistors.
(Not x86, but great for “orders of magnitude.”)

Questions

- Clock: 5 MHz $\rightarrow$ 3.5–6 GHz is about how many x?
- (Answer: About 1000x)
- “Cores: 1 $\rightarrow$ 24 is how many x?”
- (Answer: 24x)
- “RAM: 0.64 MB $\rightarrow$ 64–128 GB $\approx$?”
- (Answer: **10⁵x** range.)

Transistor Count

CPU	Year	Transistor Count	Notes/Source
Intel 8088	~1980-84	≈ 29,000	Official Number
Core i7-5775C (Broadwell, 2015)	2015	~1.9 billion	
Core i9-14900K	2025	Estimated between 20-30 billion	No public figure

Transistor count

50,000,000,000

10,000,000,000

5,000,000,000

1,000,000,000

500,000,000

100,000,000

50,000,000

10,000,000

5,000,000

1,000,000

500,000

100,000

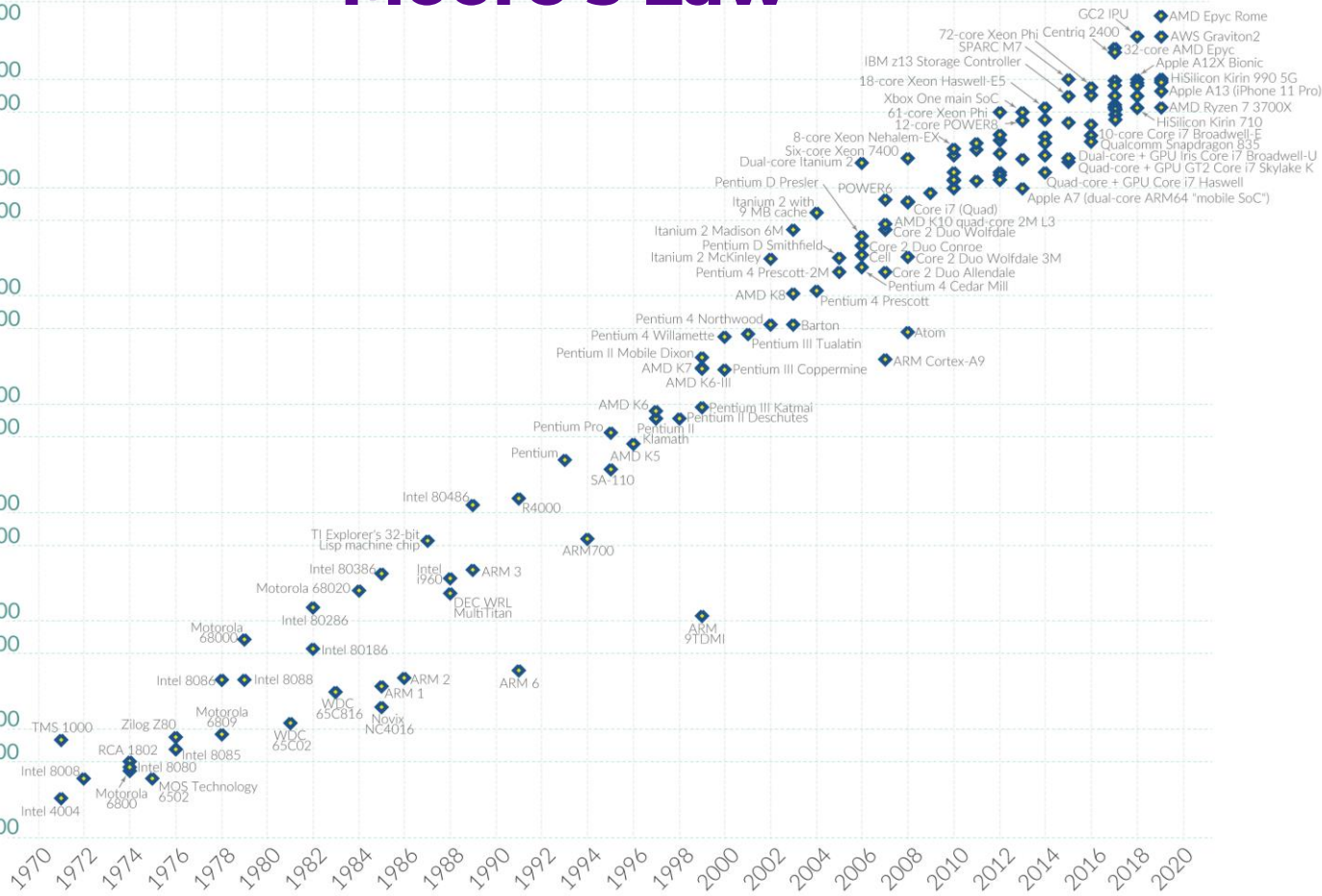
50,000

10,000

5,000

1,000

Moore's Law



The BIOS

- Responsible for the very first bit of code run on the system
- Does some simple **initialization** of basic hardware
- Also provides an **API for hardware access**
 - But this is typically not used except for the very early stages of boot now
- Knows how to **find media to boot off** of, load code from them into memory, and hand off control

What Does a CPU Do?

- Runs a simple algorithm:

pc = 0

while True:

fetch instruction at address pc in RAM

decode instruction

execute instruction

if instruction did not modify pc:

pc = pc + sizeof(instruction)

Class 4: Which one is faster?

CPU Registers

0%

RAM

0%

SSD

0%

They are all the same

0%

CPU

- **Contains a (tiny) bit of local storage (registers)**

- ~1-3 nanoseconds to access

Nanosecond = billionth of
a
second = 10^{-9}

- **Contains a little more on-chip storage (cache)**

- ~2-8 ns to access L1, 5-12 ns to access L2

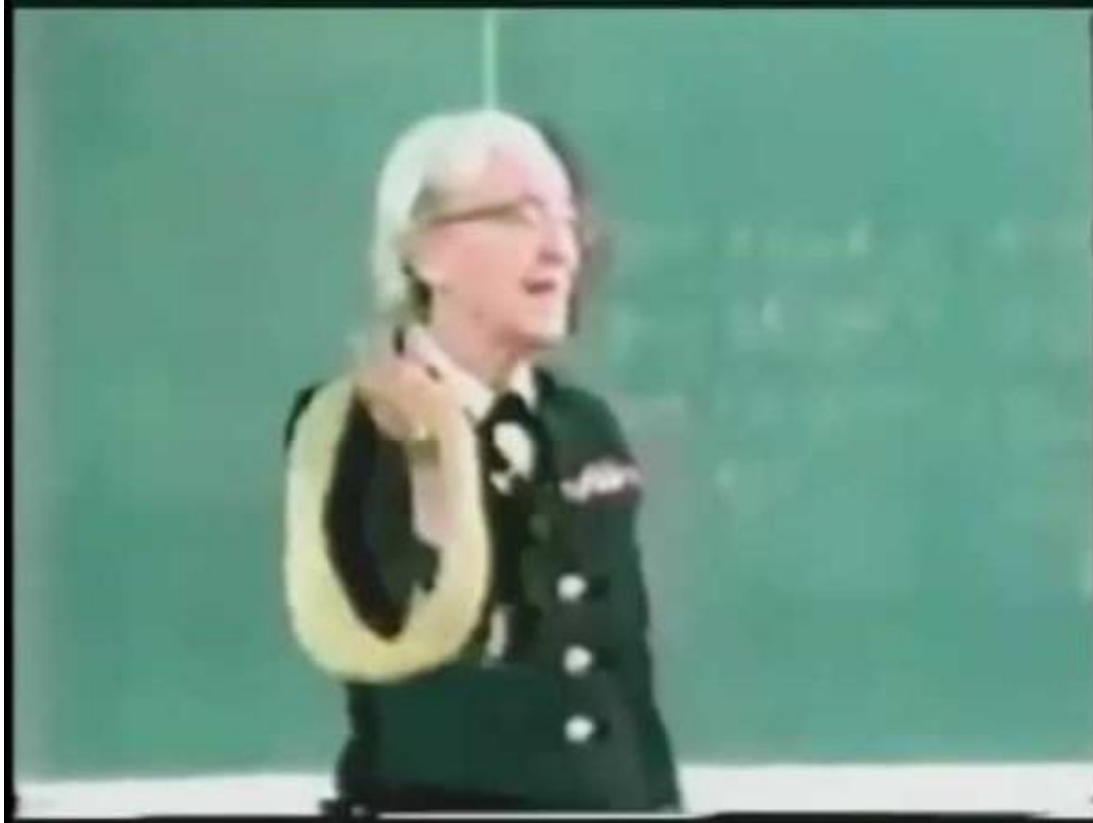
- **For comparison:**

- RAM is 10-60 ns,
- SSD is 100 μ s, (microseconds)
- disk is 3-20 **ms**

microsec = 1 millionth of a
second = 10^{-6}

millisecond = 1 thousand
of a
second = 10^{-3}

ASIDE: Grace Hopper: Nanosecond vs Millisecond.



Talking to Devices

- **Memory-mapped I/O:** A portion of the address space is mapped to a device, rather than to main memory
- **Port I/O:**
 - Special CPU instructions to talk to a fixed number of I/O ports
 - Only 1024 I/O ports defined
- **Interrupts: how devices talk back**
 - When a device wants to get the attention of the CPU, it can raise an *interrupt* (IRQ)

RAM Layout (32-bit)

Start	End	Description
0x00000000	0x0009FFFF	Low Memory
0x000A0000	0x000BFFFF	VGA Memory
0x000C0000	0x000EFFFF	16-bit Devices
0x000F0000	0x000FFFFFF	BIOS ROM
0x00100000	?	Extended Memory
...		
0xC0000000	0xFFDFFFFFF	PCI Devices
0xFFE00000	0xFFFFFFFF	High BIOS

On Linux, you can look at `/proc/iomem` for a detailed list

IA-32 Architecture

Evolutionary Design

- Starting in 1978 with 8086
- Added more and more features as time goes on
- Still support old obsolete features

Complex Instruction Set Computer(CISC)

- Many different instructions with different formats
- Only small subset found in Linux programs
- Hard to match RISC perf but Intel has done it !

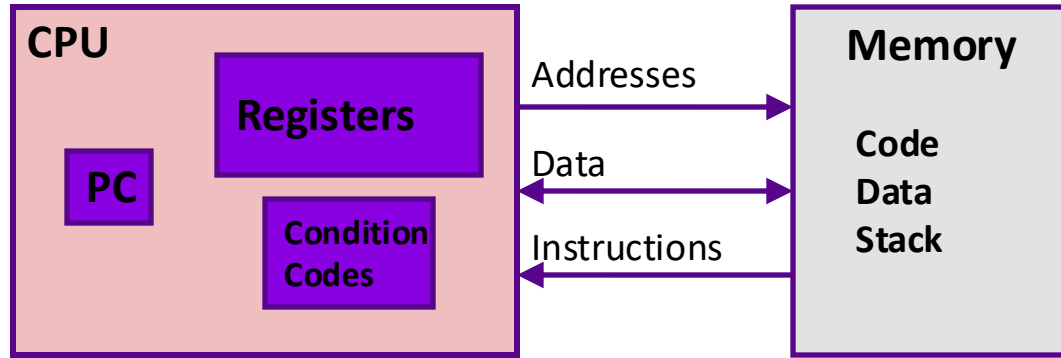
Lots of architecture improvements

- Pipelining, branch prediction, hyperthreading and multicore

From programmer's POV it has not changed substantially

- Except for intro of high perf instructions.

Assembly/Machine Code View



Programmer-Visible State

- **PC: Program counter**

- Address of next instruction

- **Registers**

- Heavily used program data

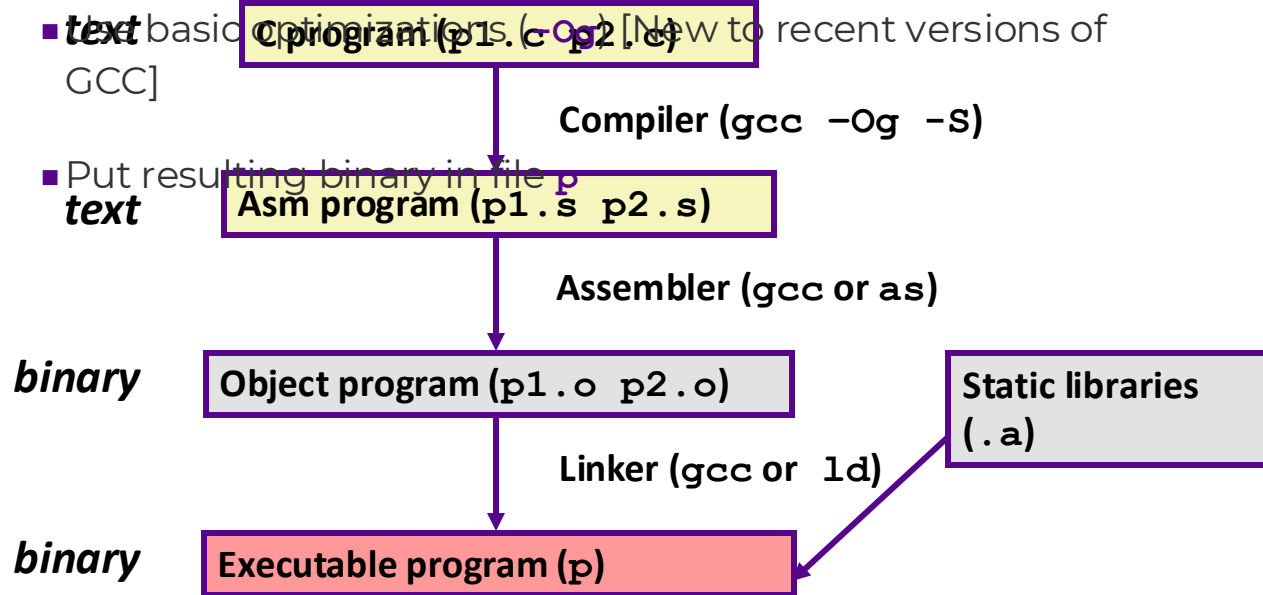
- **Condition codes**

- **Memory**

- Byte addressable array
- Code and user data
- Stack to support procedures

Turning C into Object Code

- Code in files `p1.c` `p2.c`
- Compile with command: `gcc -Og p1.c p2.c -o p`



Compiling Into Assembly

C Code

```
long plus(long x, long y);

void sumstore(long x, long y,
              long *dest)
{
    long t = plus(x, y);
    *dest = t;
}
```

Generated x86-64 Assembly

```
sumstore:
    pushq    %rbx
    movq     %rdx, %rbx
    call     plus
    movq     %rax, (%rbx)
    popq     %rbx
    ret
```

Obtain (on windows machine) with command

```
gcc -Og -fno-asynchronous-unwind-tables -S
sum.c
```

Produces file `sum.s`

Warning: Will get very different results on non-windows machines (Linux, Mac OS-X, ...) due to different versions of gcc and different compiler settings.

Modes of Operation

- **Protected Mode**
 - Native mode (windows, Linux), full features separate memory
- **Virtual 8086 mode**
 - Hybrid or protected
 - Each program has it's own 8086 computer
- **Real Address mode**
 - Native MS-DOS
- **System management mode**
 - Power management, system security, diagnostics

x86 (32-bit)

- General purpose registers:

EAX, EBX, ECX, EDX, ESI, EDI

(though some have historical uses: ECX is used for counters, ESI & EDI are source and destination for string operations)

- Special-purpose registers:

EBP (base pointer), **ESP** (stack pointer)

- Flags register: **EFLAGS**
- Segment registers: **CS, SS, DS, ES, FS, GS**
- Instruction pointer/program counter: **EIP**
- Control registers: **CR0-CR4**

Addressable Memory

- **Protected Mode**
 - 4 GB Address space
 - 32 bit addresses
- **Real address and Virtual 8086 Modes**
 - 1 MB address space
 - 20 bit addresses

32 bit General Purpose Registers

EAX
EBX
ECX
EDX

EBP
ESP
ESI
EDI

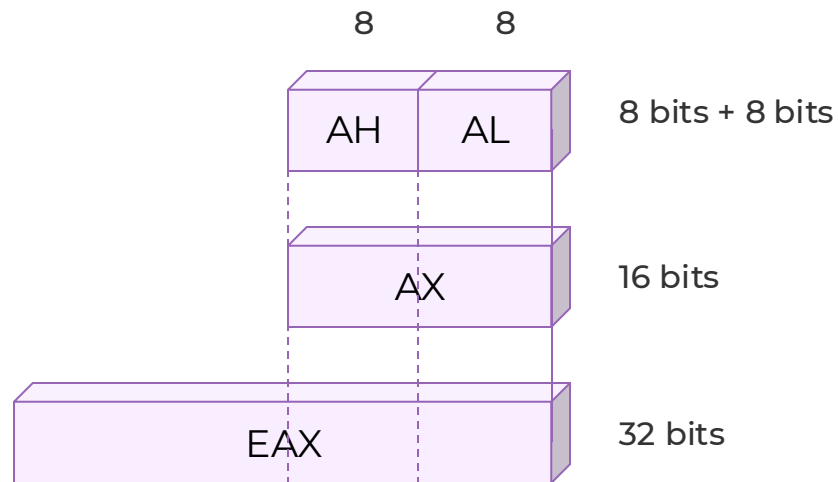
EFLAGS
EIP

16-bit Segment Registers

CS	ES
SS	FS
DS	GS

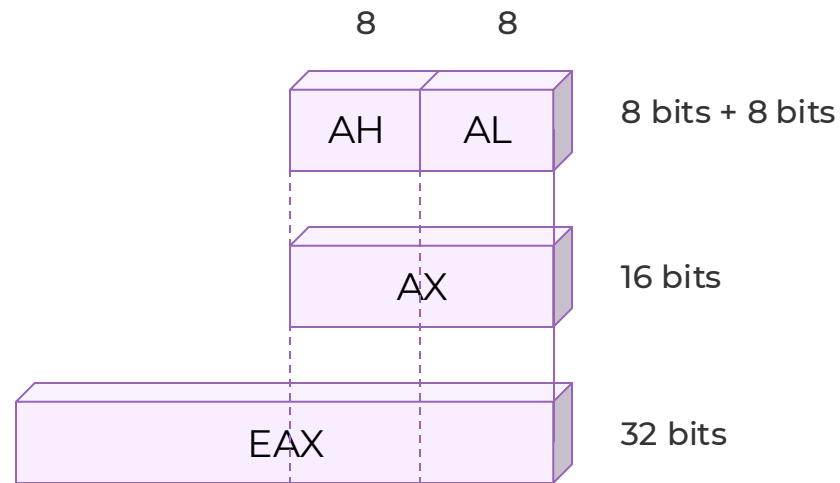
Accessing parts of registers

- Use 8 bit name, 16-bit name or 32-bit name
- Applies to EAX, EBX, ECX and EDX



Accessing parts of registers (2)

32-bit	16-bit	8-bit (high)	8 bit (low)
EAX	AX	AH	AL
EBX	BX	BH	BL
ECX	CX	CH	CL
EDX	DX	DH	DL



Index and base registers

Some registers only have a 16-bit name for their lower half (not 8 bit aliases). The 16-bit registers used only in real-address mode.

32-bit	16-bit
ESI	SI
EDI	DI
EBP	BP
ESP	SP



X86 Instructions (Refresher)

x86 Instructions

- Instructions are *variable length*
- **Conventions:**
 - regN means a register of N bits
 - mem means memory
 - constN means a constant of N bits

Note: x86 is BIG

- When they stopped printing paper manuals (c. 2010), occupied Intel manual occupied 5 volumes
- Two volumes (1300 pages as of June 2015) dedicated just to describing instructions
- We will only cover a **minimal subset** of x86 here; if you have questions about a particular instruction the Intel manuals are actually pretty good



Intel® 64 and IA-32 Architectures Software Developer's Manual
VOLUME 3B: System Programming Guide Part 2

3B



Intel® 64 and IA-32 Architectures Software Developer's Manual
VOLUME 3A: System Programming Guide Part 1

3A



Intel® 64 and IA-32 Architectures Software Developer's Manual
VOLUME 2B: Instruction Set Reference, N-Z

2B



Intel® 64 and IA-32 Architectures Software Developer's Manual
VOLUME 2A: Instruction Set Reference, A-M

2A



Intel® 64 and IA-32 Architectures Software Developer's Manual
VOLUME 1: Basic Architecture

1

x86 Assembly Syntax

- **Two "flavors":**
 - Intel syntax (what's in the processor manual)
 - AT&T syntax (what GNU as uses)
- **The difference between them:**
 - Intel: `mov eax, DWORD PTR [ebx] -> DEST, SRC`
 - AT&T: `movl (%ebx), %eax -> SRC, DEST`

We will work with **AT&T syntax**

Data Movement

- **mov** <reg>,<reg>
- **mov** <reg>,<mem>
- **mov** <mem>,<reg>
- **mov** <reg>,<const>
- **mov** <mem>,<const>

- What's missing?

Data Movement

Examples (AT&T syntax)

- Source, dest
- **movl** %eax, %edx => edx = eax
- **movl** \$0x123, %ebx => ebx = 0x123
- **movl** 0x123, %ebx => ebx = *(int *)0x123
- **movl** (%ebx), %edx => edx = *(int *)ebx
- **movl** 4(%ebx), %edx =>
 - $\text{edx} = \text{*(int *)}(\text{ebx} + 4)$

Segmentation

- Original 8086 processor had **20 address lines** (i.e., bits for specifying a memory address)
- But **only** had **16-bit registers**
- Segment registers provide a way to get the **extra 4 bits**
- Segments **also have permissions** associated with them (not discussed here)
- Modern OSes **make almost no use of segments** (and x86_64 gets rid of them in 64-bit mode)

Segmentation

- cs: **code** *segment* (modifies code fetch)
- ds: **data** *segment* (modifies data access)
- ss: **stack** *segment* (modifies ESP & EBP)
- es: **extra** *segment* (modifies string ops)
- So:
 - `movl (%ebp), %ebx` implicitly uses the 4 bits in the ss as a prefix to %ebp

Arithmetic

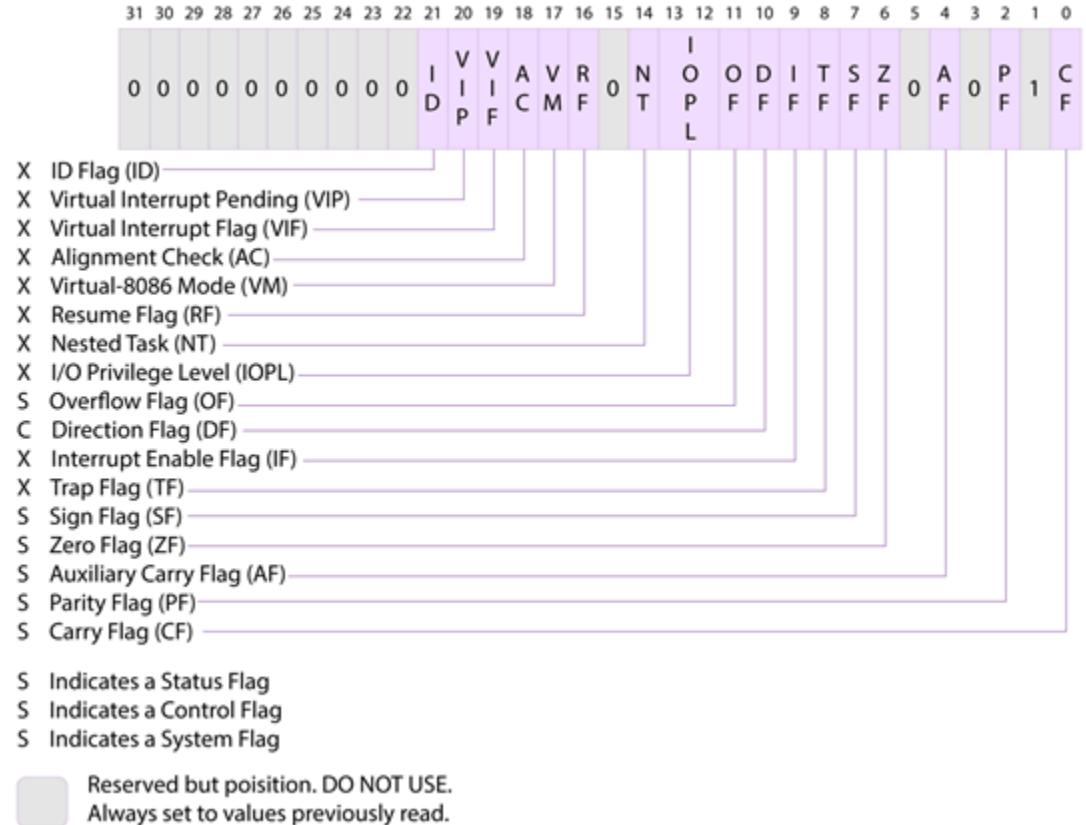
- add, sub, inc, dec
- imul, idiv
- and, or, xor
- not
- neg
- shl, shr

Logical

- test %x, %y – equivalent to **BITWISE AND** %x, %y but does not modify %y
- **cmp** %x, %y – equivalent to **sub** %x, %y but does not modify %y
- So if they don't modify the destination operand, what do they do?

EFLAGS

Arithmetic and logical instructions set bits in EFLAGS as a side effect



EFLAGS

- Conditional branches depend on the values in EFLAGS
- Also some specific instructions, e.g. `cmov`, will be executed only when flags are satisfied
- Other architectures take this even further: on ARM, almost *any* instruction can be made conditional

Conditional Branches

Instruction	Description	Signed-ness	Flags
JO	Jump if overflow		OF = 1
JNO	Jump if not overflow		OF = 0
JS	Jump if sign		SF = 1
JNS	Jump if not sign		SF = 0
JE JZ	Jump if equal Jump if zero		ZF = 1
JNE JNZ	Jump if not equal Jump if not zero		ZF = 0
JB JNAE JC	Jump if below Jump if not above or equal Jump if carry	unsigned	CF = 1
JNB JAE JNC	Jump if not below Jump if above or equal Jump if not carry	unsigned	CF = 0
JBE JNA	Jump if below or equal Jump if not above	unsigned	CF = 1 or ZF = 1
JA JNBE	Jump if above Jump if not below or equal	unsigned	CF = 0 and ZF = 0

Instruction	Description	Signed-ness	Flags
JL JNGE	Jump if less Jump if not greater or equal	signed	SF <> OF
JGE JNL	Jump if greater or equal Jump if not less	signed	SF = OF
JLE JNG	Jump if less or equal Jump if not greater	signed	ZF = 1 or SF <> OF
JG JNLE	Jump if greater Jump if not less or equal	signed	ZF = 0 and SF = OF
JP JPE	Jump if parity Jump if parity even		PF = 1
JNP JPO	Jump if not parity Jump if parity odd		PF = 0
JCXZ JECXZ	Jump if %CX register is 0 Jump if %ECX register is 0		%CX = 0 %ECX = 0

Unconditional Branch

- **jmp: unconditional jump**
 - **jmp** <const> – jump to constant address
 - **jmp** <mem> – dereference mem and jump
 - **jmp** <reg> - jump to address in register

String

- None of them take **an explicit operand**; they work with implicit operands.
 - Source is **DS:SI**
 - Destination: **ES:DI**
- **lodsb, stosb**: loads or stores a byte from %esi or %edi and increments %esi/%edi
- **movsb**: move byte from (%esi) to (%edi) and increments both
- **scasb**: compares %(es:edi) with %al and sets flags, then increments %edi
- String operations can be used with the repCC prefix, which repeats the instruction while a condition CC is true

System

- Many instructions control system behavior
- **int** – executes a *software interrupt* (traps into the kernel)
- **iret** – return from an interrupt
- **sysenter/sysret** – newer way to enter/leave kernel mode

Labels

- **Data label**
 - must be unique
 - example: `myArray` `BYTE 10`
- **Code label (ends with a colon)**
 - target of jump and loop instructions
 - example: `L1: mov ax, bx`

...

`jmp L1`

Fill in the blanks

Address	Value	Register	Value
0x100	0xFF	%rax	0x100
0x104	0xAB	%rcx	0x1
0x108	0x13	%rdx	0x3
0x10C	0x11		

Operand	Value	Comment
%rax		
0x104		
\$0x108		
(%rax)		
4(%rax)		

Fill in the blanks

Address	Value	Register	Value
0x100	0xFF	%rax	0x100
0x104	0xAB	%rcx	0x1
0x108	0x13	%rdx	0x3
0x10C	0x11		

Operand	Value	Comment
%rax	0x100	Register
0x104	0xAB	Absolute Address
\$0x108	0x108	Immediate
(%rax)	0xFF	Address 0x100
4(%rax)	0xAB	Address 0x104

Assembly: Jumps

Instruction	Effect
jmp	Always jump
je/jz	Jump if eq / zero
jne/jnz	Jump if !eq / !zero
jg	Jump if greater
jge	Jump if greater / eq
jl	Jump if less
jle	Jump if less / eq

Push/Pop

- Recall: **ESP** is stack pointer
- On x86 the **stack grows downward** – putting something on the stack decrements the stack pointer
- The **push** instruction **subtracts 4 from ESP**, then **writes to the memory** at address ESP

Function Calls

- Supports creation of functions
- **call <f>**: push the address of next instruction on the stack and jump to f
- Example:

```
1 1000 call <f>           // pushes 5 and jumps to f
2 1005 movl %eax, %ebx
```

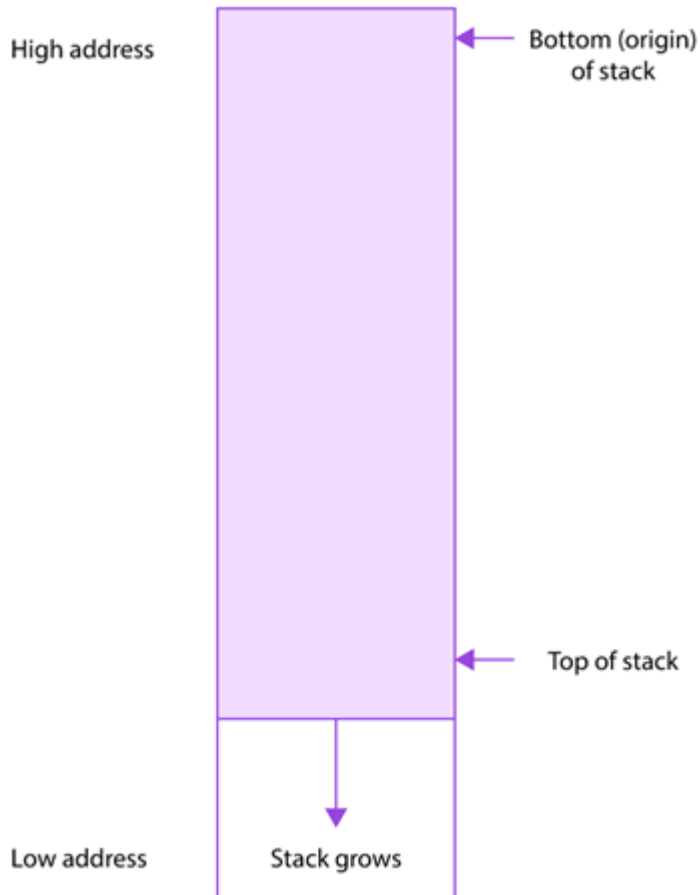
- **ret**: pop return address off the stack and jump to it

Stack Frames

- x86 programs use the stack to support function calls.
- The machine uses the stack to:
 - Pass arguments
 - store return information
 - Save registers
 - Local Storage
- Therefore, to use arguments and return information, we need to deal with the stack.
- The portion of memory used in the stack for a single procedure call is referred to as a **stack frame**.

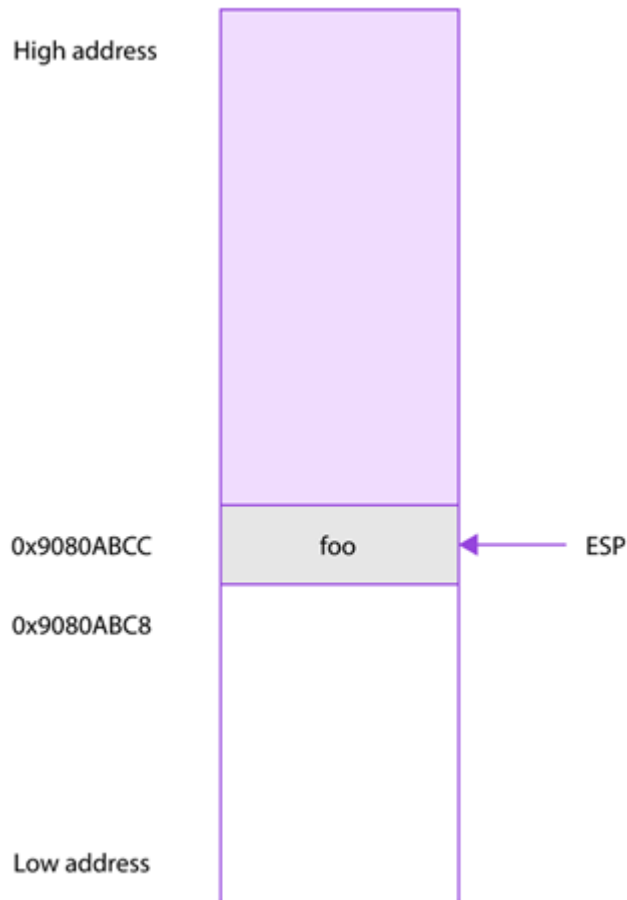
The stack in x86

- The stack in x86 grows down!
- When we say “top of the stack” on x86 we actually mean the lowest address in the memory area occupied by the stack.
- This might be “**unnatural**” but keep it in mind.



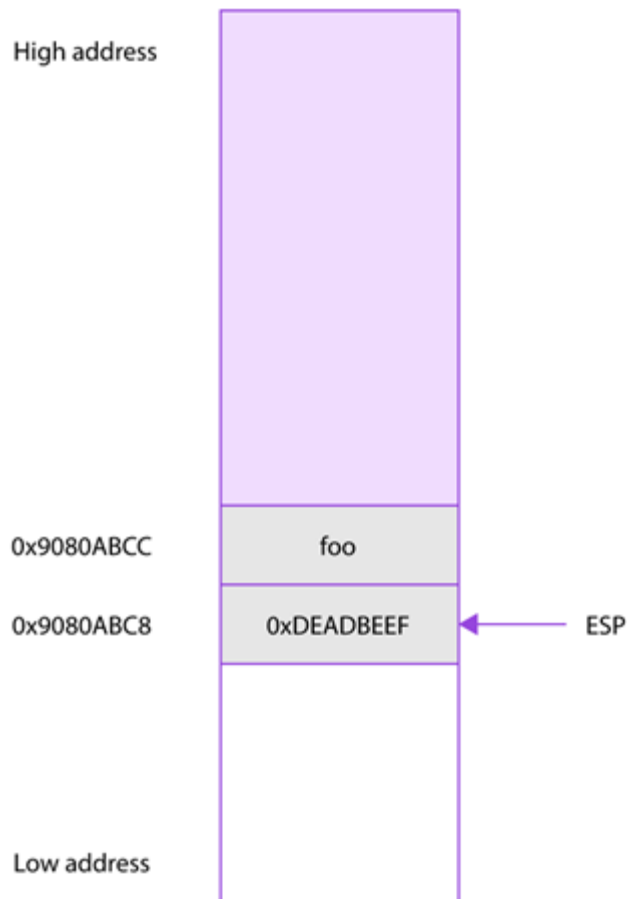
Pushing and Popping data

- We **push** data with the **stack pointer**
- 0x9080ABCC top of stack
- The word at that address has “foo”
- ESP has the address 0x9080ABCC



Pushing and Popping data

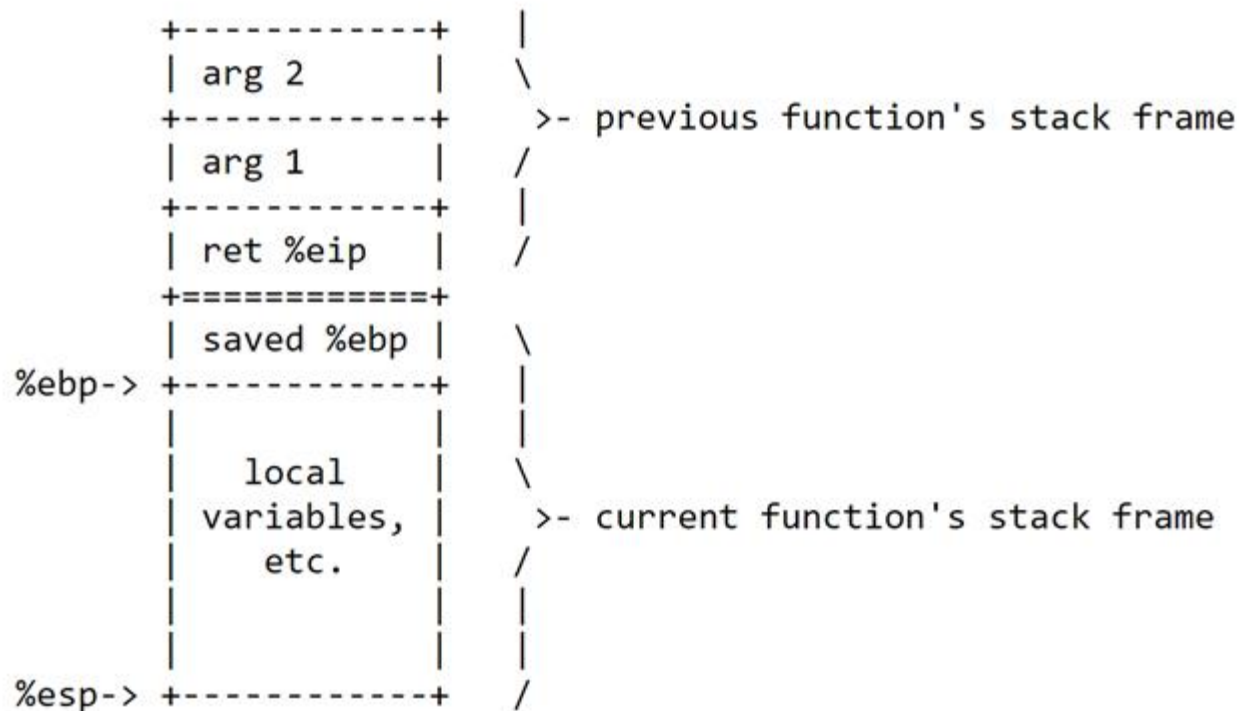
- To push new data we use the **push** instruction.
- push
 - Decrement esp by 4
 - Store its operand in the location esp points to
- Example:
 - Push eax



Stack Frames

- The topmost stack frame is delimited by two register pointers:
 - **%ebp** is the frame pointer (base)
 - **%esp** is the stack pointer
- **So stack arguments can be accessed with `%(ebp)`**

Stack Usage



Function Prologue & Epilogue

- The need to set up a stack frame gives rise to a common function prologue:

```
push %ebp           // save old value of ebp
mov %esp, %ebp      // set base pointer
sub $X, %esp        // reserve space for locals
```

- At the end, we can clean up with just the epilogue

```
mov %ebp, %esp      // restore stack pointer
pop %ebp            // restore base pointer
ret                 // return pops return address
```

Object Code

Code for sumstore

0x0400595:

0x53

0x48

0x89

0xd3

0xe8

0xf2

0xff

0xff

0xff

0x48

0x89

0x03

0x5b

0xc3

- Total of 14 bytes
- Each instruction 1, 3, or 5 bytes
- Starts at address 0x0400595

Assembler

- Translates **.s** into **.o**
- Binary encoding of each instruction
- Nearly-complete image of executable code
- Missing linkages between code in different files

Linker

- Resolves references between files
- Combines with static run-time libraries
- E.g., code for **malloc**, **printf**
- Some libraries are *dynamically linked*
- Linking occurs when program begins execution